Day 9 Student Notes: Claude Code + GitHub

> **Today in one line:** Claude Code does the Git work (branch, commit, push, pull request). **You read, approve and decide.**

The path from idea to main

```
idea -> Claude builds it -> commit -> push to GitHub (private)
 -> branch + pull request -> you review -> merge to main
```

Three ways to connect Claude and GitHub

Way	What it is	Good for
1. Claude Code + gh (today)	Claude Code on your laptop runs git and the GitHub tool gh	Build, commit, push, open and merge PRs
2. GitHub connector in the Claude app	Settings, Connectors. Lets Claude read your repos from chat	Asking about code and PRs
3. @claude on GitHub	A GitHub Action. Comment @claude on an issue or PR. Set up with /install-github-app	Teams (needs repo admin and a secret)

One-time setup

```
git --version
gh --version
gh auth login
gh auth status
```

gh auth login: choose **GitHub.com**, **HTTPS**, authenticate Git **Yes**, **Login with a web browser**. Copy the one-time code and approve in the browser. Never paste a token into Claude or a file. Windows without gh: **winget install --id GitHub.cli**, then restart VS Code.

CLAUDE.md: the team's Git rules for Claude

```
# Git rules
- Never push to main. Use branches.
- Branch names: feature/short-name
- Commit messages: type(area): summary, then why
- PR descriptions: Summary, Changes, Risk, How to test
- Never force push
- Ask me before any push or merge
- Never commit secrets
- Repos are private
```

It is a file in the project folder. Claude Code reads it at the start, so the whole team gets the same behaviour.

The demo, step by step (prompts are in Day09_Prompts_To_Paste.txt)

1. **Build:** Claude creates README.md, services.txt, health_check.sh. The script prints OK, OK, FAIL (the FAIL is on purpose).
2. **Rules:** Claude writes CLAUDE.md.
3. **Repo:** Claude sets up Git, creates a **private** GitHub repo with gh and pushes. Check on github.com.
4. **Branch:** feature/add-timeout, three small commits, main untouched. Check with **git log --oneline** and **git diff main**.
5. **Pull request:** Claude pushes the branch and opens a PR with Summary, Changes, Risk, How to test. Open it with **gh pr view --web**.
6. **Review, then merge:** Claude reviews the diff. You read it. Only then Claude squash-merges and deletes the branch.
7. **Release notes:** Claude writes a changelog from the history, on its own branch and PR.

Fixing history safely

Tool	What it does
Squash	Joins many small commits into one
Rebase	Replays your branch on top of newer main
Cherry-pick	Copies one commit to another branch

> **Safety rules:** make a backup branch first. Only change your own branch that is not pushed. Never rewrite main. If you must force push your own branch, use **--force-with-lease**.

What you approve, and what you never approve

Say yes after you read	Stop and think
git status, diff, log	Push to main
Commit on a feature branch	Force push, reset --hard
Push a feature branch	Public repo, or keys and .env in files
Open a PR	Merge without reading the diff
	Any command you do not understand

If unsure, ask Claude: "Explain this command and what could go wrong."

Key words

Word	Meaning
GitHub	A website that stores Git repositories and pull requests
gh	GitHub's command-line tool. Claude uses it to create repos and PRs
Remote / origin	The copy of the repo on GitHub
Push	Send your commits to GitHub
Squash merge	Merge a PR as one single commit
CLAUDE.md	Rules Claude Code reads from your project folder
Changelog	A list of what changed in each version

Check yourself

1. Why do we run gh auth login in the browser instead of pasting a token?
2. What is CLAUDE.md for?
3. Why do we make a branch instead of changing main?
4. What must you do before telling Claude to merge?
5. What do you make before squashing or rebasing?

Answers

1. Tokens in chat or files can leak. The browser login keeps the secret out of sight.
2. It tells Claude Code your team's rules, such as branch names and never push to main.
3. Main stays safe, and the change can be reviewed in a PR first.
4. Read the diff and the PR description.
5. A backup branch.

Take-home task (30 minutes)

1. Pick a tiny idea of your own (for example a script that prints today's date and disk space). New folder, new **private** repo.
2. Use the flow: build, CLAUDE.md, repo, branch, PR, review, merge.
3. Add a changelog PR from the history.
4. Bring your PR link and one thing in Claude's work you changed or rejected.

Private repos only. No real company code, passwords or keys.

> **Next class, Day 10:** Checkpoint project: Linux and Git together with Claude.